

Funciones de libpcap (en C)

Práctica 2 — Jiménez Cázares Ángel Rodrigo - 321173579

Aquí deajo investigadas y resumidas las 5 funciones. Todo esto es para Linux (en mi caso Debian), usando la librería libpcap, que es la que se usa para capturar y mandar tráfico crudo directo desde la tarjeta de red.

Para que estas funciones sirvan hay que instalar la librería con: `sudo apt install libpcap-dev` y compilar con `-lpcap`. Casi siempre hay que correr el programa con `sudo` porque meterte a la tarjeta de red en modo raw pide permisos de root.

1. pcap_open_live

Esta es la que abre la interfaz de red (tipo eth0) para empezar a escuchar el tráfico que pasa en vivo.

```
pcap_t *pcap_open_live(const char *device, int snaplen, int promisc, int to_ms, char *errbuf);
```

- `device`: el nombre de la interfaz que vas a usar (se puede ver con `ip a` en la terminal).
- `snaplen`: cuántos bytes como máximo va a agarrar de cada paquete (normalmente se pone 65535 para que se lleve el paquete completo).
- `promisc`: si tiene un valor diferente de 0, intenta activar el modo promiscuo, permitiendo capturar también tramas que llegan a la interfaz aunque no estén dirigidas específicamente a su dirección MAC.
- `to_ms`: el tiempo que espera (en milisegundos) antes de que se le acabe la paciencia leyendo.
- `errbuf`: un espacio en memoria (mínimo `PCAP_ERRBUF_SIZE`) donde se guarda el mensaje de error si algo sale mal.

Lo que regresa es un puntero `pcap_t*`, que viene siendo el "identificador" de esa sesión de captura, y que vas a necesitar para todas las demás funciones. Si algo falló, regresa `NULL`.

2. pcap_sendpacket

Esta manda un paquete que tú ya armaste a mano (con su encabezado Ethernet y todo) directo por la interfaz de red, sin pasar por toda la pila de red normal del sistema.

```
int pcap_sendpacket(pcap_t *p, const u_char *buf, int size);
```

- `p`: el identificador que te dio `pcap_open_live`.
- `buf`: el arreglo de bytes con el paquete completo listo para mandarse.
- `size`: cuántos bytes mide ese paquete.

Regresa 0 si todo salió bien, o -1 si hubo un error (ahí es cuando entra `pcap_geterr` para saber qué pasó).

3. pcap_loop

Esta pone al programa a escuchar sin parar, y cada que llega un paquete ejecuta automáticamente una función que tú defines (un callback), hasta que le digas que pare o llegue al límite de paquetes que le pusiste.

```
int pcap_loop(pcap_t *p, int cnt, pcap_handler callback, u_char *user);
```

- `p`: el identificador de la sesión de captura.

- cnt: cantidad de paquetes que se quieren procesar antes de terminar. Si se utiliza -1, el ciclo continúa procesando paquetes hasta que ocurra un error o se detenga explícitamente.
- callback: la función que programaste tú, la cual se dispara solita cada vez que cae un paquete nuevo.
- user: un puntero opcional por si quieres mandarle datos extra a tu función callback.

Regresa 0 si terminó normal, o un número negativo si hubo algún error o se interrumpió a la mitad.

4. pcap_close

(Nota: en el PDF de la práctica aparece escrito como "cap_close", pero el nombre correcto de la función en libpcap es pcap_close, seguramente fue un error de dedo.)

Esta función cierra la sesión de captura y libera los recursos asociados al manejador `pcap_t`. Debe llamarse cuando el programa ya no necesite utilizar la sesión.

```
void pcap_close(pcap_t *p);
```

- p: el identificador de la sesión que quieres cerrar.

No regresa nada, es void.

5. pcap_geterr

Esta te regresa un mensaje de texto entendible explicando cuál fue el último error que ocurrió en la sesión, súper útil para saber qué está fallando cuando alguna otra función regresa un código de error.

```
char *pcap_geterr(pcap_t *p);
```

- p: el identificador de la sesión donde pasó el error.

Regresa un puntero a un texto (que ya lo maneja la librería internamente, no hay que liberarlo a mano) con la descripción de qué salió mal.

Resultados

Para comprobar el funcionamiento de los programas se utilizaron dos terminales. En una terminal se ejecutó el receptor, el cual quedó escuchando las tramas en la interfaz `wlp1s0`. En la otra terminal se ejecutó el emisor enviando el mensaje **"Hola Redes2027-1"**.

El emisor logró enviar correctamente la trama y el receptor identificó la trama con EtherType `0x88B5`, mostrando las direcciones MAC de origen y destino, así como el mensaje contenido en el payload.

Como se observa en las siguientes capturas, el mensaje enviado por el emisor fue recibido correctamente por el receptor.

```
rodrigo@debian: ~/Escritorio/Redes de Computadoras/Practica2
rodrigo@debian:~/Escritorio/Redes de Computadoras/Practica2$ sudo ./receptor
Interfaz wlp1s0 abierta correctamente
Escuchando en wlp1s0...

--- Trama recibida ---
MAC destino: ff:ff:ff:ff:ff:ff
MAC origen: 5c:61:99:2c:75:99
EtherType: 0x88b5
Mensaje: Hola Redes2027-1
```

```
rodrigo@debian:~/Escritorio/Redes de Computadoras/Practica2$ sudo ./emisor "Hola Redes2027-1"
Interfaz wlp1s0 abierta correctamente
Mensaje guardado en la trama: Hola Redes2027-1
Trama enviada correctamente
rodrigo@debian:~/Escritorio/Redes de Computadoras/Practica2$
```

Conclusión

En esta práctica pude entender de una manera más clara cómo funciona la comunicación en la capa 2 del modelo OSI, ya que anteriormente conocía algunos de estos conceptos más que nada de forma teórica. Al realizar los programas pude ver de manera práctica cómo se construye una trama Ethernet y qué información contiene, como las direcciones MAC de origen y destino, el EtherType y el payload.

También aprendí a utilizar algunas de las funciones que ofrece la librería libpcap para trabajar directamente con las tramas de red. Con el programa emisor pude construir una trama y colocar dentro de ella un mensaje como payload, mientras que con el receptor pude capturar esa misma trama, identificarla mediante su EtherType y recuperar el mensaje que había sido enviado.

Finalmente, al ejecutar los dos programas pude comprobar que la trama se enviaba y recibía correctamente, ya que el receptor mostró el mismo mensaje que había enviado desde el emisor. En general, considero que esta práctica me ayudó a relacionar mejor los conceptos vistos sobre la capa de enlace con lo que realmente sucede cuando se envía información a través de una red.